HSE, VK

02/02/2026

Matrix-Factorisation Based Methods.

Alexander Tarakanov

Outline

Limitations of Classical Collaborative Filtering

User-based Collaborative Filtering

- User similarity is unstable in sparse data
- Neighborhoods change rapidly over time
- Poor scalability with large user base

Item-based CF (heuristic similarity)

- Similarity measures are hand-crafted (cosine, Jaccard)
- Captures only local co-occurrence patterns
- No global optimization objective

Matrix Factorization

- Latent factors lack interpretability
- Sensitive to hyperparameter tuning
- Cold-start items poorly handled

Item–Item Propagation Perspective

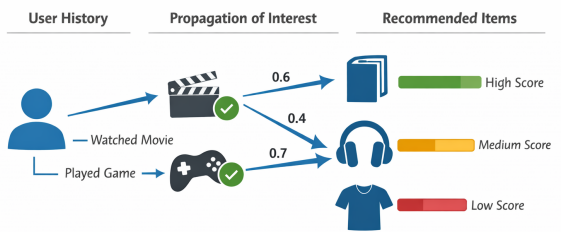
Core idea

- B is a **learned item–item propagation operator**
- Replaces heuristic similarity with data-driven learning
- Preference signals propagate between related items

Why this helps

- Robust to sparsity and implicit feedback
- Item structure is more stable than user structure
- Global objective enforces consistency
- Interpretable and scalable recommendation mechanism

$$\hat{X} = XB$$



Outline

SLIM. Sparse Linear Models.

User-Item intercation data is stored in X
objective - miminise loss-function:

$$\mathcal{L}(B) = \frac{1}{2}||X - XB||_F^2 + \lambda_1||B|| + \lambda_2||B||^2$$

Subjected to the following constraints:

$$\begin{cases} B_{ij} \geq 0, \text{ for any i and j,} \\ B_{ii} = 0, \text{ for any i.} \end{cases}$$

Recommender score is simply:

$$s_r = XB$$

SLIM: Advantages and Limitations

Advantages

- Learns item–item relations directly from data
- Interpretable sparse propagation matrix
- Naturally handles implicit feedback
- Robust to noisy user interactions
- Stable item-based structure

Limitations

- Requires solving many sparse regression problems
- Training is computationally expensive
- Memory cost grows with number of items
- Limited scalability to very large catalogs
- Sparsity may miss global structure

SLIM trades scalability for interpretability and robustness.

Outline

EASE. Embarrasingly Shallow AutoEncoders.

Miminise loss-function

$$\mathcal{L}(B) = \frac{1}{2}||X - XB||_F^2 + \frac{1}{2}\lambda||B||_F^2 + \gamma \text{diag}(B)$$

Zero diagonal is the only constraint.

EASE admits the exact solution in terms of the the inverse of the following matrix:

$$H = \left(X^\top X + \lambda I\right)^{(-1)}$$

Lagrange Multipliers formalism allows one to compute the solution explicitly:

$$B_{ij} = -H_{ij}/H_{jj}, \text{ for } i \neq j.$$

Recommender score is simply:

$$s_r = XB$$

EASE. Derivation 1.

Loss-function:

$$\mathcal{L}(B) = \frac{1}{2}||X - XB||_F^2 + \frac{1}{2}\lambda||B||_F^2 + \gamma\text{diag}(B) =$$
$$= \frac{1}{2}\text{tr}\big((B^\top X^\top - X^\top)(XB - X)\big) + \frac{1}{2}\lambda\text{tr}(B^\top B) + \gamma\text{diag}(B)$$

Minimum of loss function is equivalent to zero value of first derivative. The latter can be expressed in terms of perturbations δB, $\delta\gamma$ in Lagrange formalism:

$$\mathcal{L}(B + \delta B) = \mathcal{L}(B) + o(||\delta B, \delta\gamma||)$$

Therefore:

$$\text{tr}\big(\delta B^\top (X^\top XB - X^\top X + \lambda B)\big) + \gamma\text{diag}(\delta B) + \delta\gamma\text{diag}B$$

This leads to the following equation on B:

EASE. Derivation 2.

Notation:

$$H = \left(X^\top X + \lambda I\right)^{(-1)}$$

Expression for B matrix:

$$B = H\left(X^\top X - \text{diagMat}(\gamma)\right) = H\left(X^\top X + \lambda I - \lambda I - \text{diagMat}(\gamma)\right) = H\left(H^{(-1)} - \text{diagMat}(\gamma + \lambda)\right)$$

Finally, B can be computed as following:

$$B = I - H\text{diagMat}(\gamma + \lambda)$$

γ is adjusted to meet the constaint: $\text{diag}(B) = 0$

Outline

SANSA. Scalable Approximate NonSymmetric Autoencoder for Collaborative Filtering.

The main issue with EASE is scalability: complexity rapidly increases as number of items grow.

The reason: dense matrix B.

Solution: sparse matrix techniques to reduce the computational cost of calculation of $\left(X^\top X + \lambda I\right)^{(-1)}$.

The idea of the approach is to approximate the inverse of $A = X^\top X + \lambda I$:

- introduce the permutation P
- Sparse Cholesky decomposition for: $PAP^\top \approx LDL^\top$
- Approximate Sparse Inverse: $K \approx L^{(-1)}$
- $K \approx L^{(-1)}$
- $W = KP$
- $Z_0 = D^{(-1)}W$
- $r = \text{diag}(W^\top Z_0)$
- Z is derived from Z_0 by scaling the columns of Z_0 by $-1/r$
- B is approximated by the product $W^\top Z$

Rankong is computed so that the ranking score is the following:

$$s_r = XB$$

SANSA: Why Cholesky Decomposition?

Recall the EASE matrix:

$$P = A^\top A + \lambda I$$

- P is symmetric and positive definite
- Direct inversion of P is infeasible for large item sets

Key idea of SANSA

- Avoid explicit matrix inversion
- Factorize P using Cholesky decomposition
- Approximate the inverse through sparse triangular solves

Cholesky allows us to replace inversion by efficient linear solves.

Cholesky Decomposition

Theorem (Cholesky Decomposition)
If $P \in \mathbb{R}^{n \times n}$ is symmetric positive definite, then

$$P = LDL^\top$$

where:

- L is unit lower triangular
- D is diagonal with positive entries

Algorithm (conceptual)
For $j = 1, \ldots, n$:

- Compute diagonal:

$$D_{jj} = P_{jj} - \sum_{k<j} L_{jk}^2 D_{kk}$$

- Compute off-diagonal entries:

$$L_{ij} = \frac{1}{D_{jj}} \left(P_{ij} - \sum_{k<j} L_{ik} L_{jk} D_{kk} \right), \quad i > j$$

Cholesky exploits symmetry and positive definiteness of P.

SANSA: Using Cholesky for Approximate Inversion

Step 1: Sparse Cholesky factorization

$$P \approx LDL^\top$$

- Factorization is computed approximately
- Sparsity is preserved via fill-in control

Step 2: Approximate inverse

- Instead of computing P^{-1}, SANSA approximates:

$$P^{-1} \approx L^{-T} D^{-1} L^{-1}$$

- Triangular solves are efficient and scalable

Outcome

- Avoids dense matrix inversion
- Preserves EASE semantics
- Enables large-scale item–item propagation

SANSA turns matrix inversion into sparse linear algebra.

Outline

SLIM vs EASE vs SANSA

Property	**SLIM**	**EASE**	**SANSA**
Propagation operator B	Sparse	Dense	Approx. sparse
Learning principle	Sparse regression	Ridge regression	Numerical approximation
Closed-form solution	No	Yes	Approximate
Scalability	Medium	Low	High
Memory usage	Sparse	Dense $O(n^2)$	Controlled sparse
Interpretability	High	Medium	Medium
Optimization objective	Explicit	Explicit	Same as EASE
Numerical approximation	No	No	Yes
Graph interpretation	Sparse item graph	Dense item graph	Spectral / sparse graph
Typical use case	Medium-scale systems	Strong baseline	Large-scale systems

All three methods learn an item–item propagation operator; they differ in how it is computed and scaled.